Investigating the use of Reinforcement Learning Algorithms in Traffic Environments

Abdul Muhaymin Abdul Hafiz[1]

BSc Computer Science

Supervisor: Prof. Graham Roberts

Submission date: Day Month Year

[1]**Disclaimer:** This report is submitted as part requirement for the BSc Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Summarise your report concisely.

Contents

1 Introduction **2**

1.1 The Problem . . . 2

1.2 Aims and Objectives . . . 2

1.3 Report Overview . . . 3

2 Simulated Environment **4**

2.1 OpenStreetMap . . . 4

2.2 Simulation of Urban MObility . . . 5

2.3 Route Generation . . . 5

2.3.1 Binomial Distribution . . . 7

2.3.2 Poisson Distribution . . . 7

2.4 Summary . . . 8

3 Reward System **9**

3.1 Existing Traffic Lights . . . 9

4 etc. **10**

5 Conclusions **11**

5.1 Achievements . . . 11

5.2 Evaluation . . . 11

5.3 Future Work . . . 11

A Other appendices, e.g., code listing **13**

Chapter 1

Introduction

1.1 The Problem

Traffic lights are ubiquitous signalling devices used to control the flow of vehicle and pedestrian traffic, primarily for safety and better regulation of traffic. First introduced in 1868 on Parliament Square in London [Tha17], there have been a wide range of designs created with the most common being a three-colour design – that is green, amber, and red.

The three-colour de jure standard typically uses an internal clock to cycle through and control the duration of each signal phase in a pre-programmed pattern. Whilst this is an obvious approach, the fixed timer takes for granted that traffic demand will remain relatively constant over time, disregarding finer granularity and variations in conditions.

In an endeavour to improve upon this, advanced traffic light systems equipped with sensors are progressively being introduced. These systems dynamically adapt signal timings in response to observed traffic conditions by using rule-based logic and manually designed heuristics. This approach offers improvements in waiting times and vehicle throughput [FHW17], however it relies on predefined rules defined by domain experts that may fail to generalise and perform optimally in all conditions.

1.2 Aims and Objectives

The overall aim of this dissertation will be to perform a detailed analysis and evaluation of how an intelligent agent may be used to hopefully improve upon existing traffic light systems. Rather than explicitly defining behaviour in different traffic scenarios as before, we will observe how optimal decision-making policies may be learnt independently through the use of reinforcement learning algorithms.

Based on this aim, the following objectives are defined:

1. Define an environment in which our simulations and experiments will be conducted.
2. Build an abstracted computer representation of the area.
3. Review the traffic light systems currently in use.
4. Develop our own intelligent agents progressively, going through various reinforcement learning algorithms to see what improvements are made.

5. Consider the impact on both pedestrians and vehicles, and the effects of changing the road design in itself as a means of making improvements.

6. Evaluate the performance of our intelligent agents.

7. Finally, assess the feasibility of our agents and what factors would need to be considered in a tangible implementation.

Time is a currency that once spent cannot be regained. Traffic lights govern our lives to the extent that saving mere fractions of a second at the individual level can compound to an indescribable amount saved when considering the aggregate. By leveraging intelligent agents, this research hopes to take traffic lights a step further – regardless of how big that step may be.

1.3 Report Overview

!!! MUST REDO SINCE THE STRUCTURE HAS CHANGED !!!

We will begin with defining the environment in which we will be conducting our traffic experiments, and then building out this environment using available open source tools and libraries. Whilst accuracy and scope of the environment are important, they will not take precedence over the implementation and analysis of the intelligent agents themselves.

Following this, the context and theory behind reinforcement learning itself will be explored, as well as definitions and terminology that will be used throughout the dissertation. These foundations will be necessary when explaining the motivation behind decisions made later on.

Next we will discuss the creation of an application programming interface that will enable us to connect to our environment for the purposes of observation and control. Based on the default environment and traffic light systems, we will also define a reward which will be used to evaluate performance.

Upon making our environment, we will implement and explore different reinforcement learning algorithms our intelligent agent may use – namely Q-learning and Deep Q-learning. These discussions will be supplemented with the theory regarding how each works, as well as the improvements and limitations proposed by each. Further derivatives of these algorithms and the advantages they bring will also be briefly touched upon, however they will not be implemented.

Continuing from this, a novel approach in which multiple intelligent agents interface together through the use of a network layer will be detailed and implemented. The limitations imposed by such an approach will be considered, as well as how it may be implemented in practice outside of the simulated environment setting.

Finally, after testing and evaluating the different approaches in unforeseen traffic scenarios, we will draw our overall conclusions. Feasibility of such a project will be discussed, as well as interesting technical challenges that were encountered. Future work and limitations to consider will then be commented on.

Chapter 2

Simulated Environment

Before we get to our intelligent agents, we must first construct an environment where the agent can receive input – or state, rather – and perform actions. For the sake of simplicity this environment will be constrained to the streets surrounding University College London's (UCL) Bloomsbury Campus. This area provides scope sufficient for the agents to train well and generalise to other situations.

2.1 OpenStreetMap

OpenStreetMap is a map database maintained by a community of volunteers. Using their data obtained from ariel photo imagery, satellite imagery, geodata sources, as well as manual community additions [HW08], we can extract a portion of UCL's Bloomsbury Campus[1], also complete with data such as traffic lights and crossings.

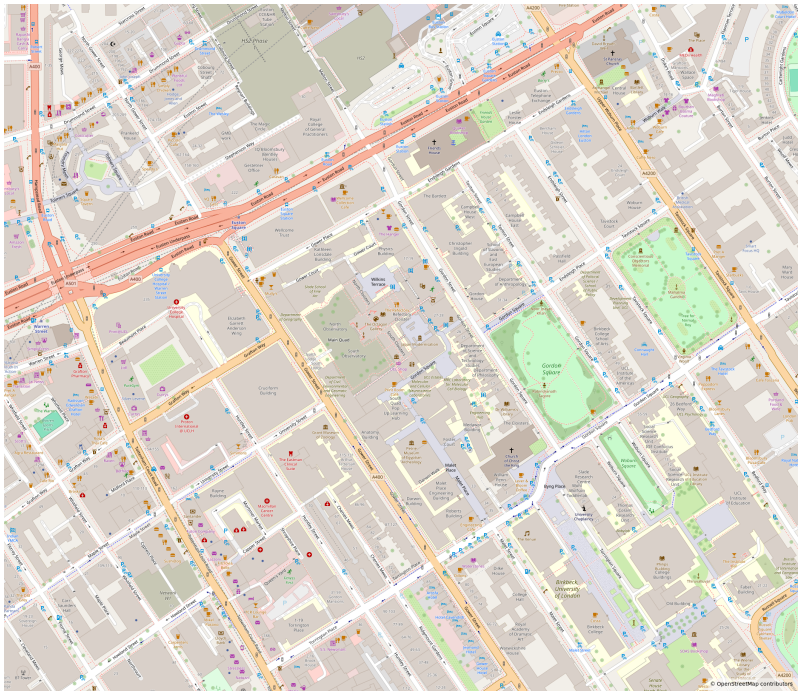


Figure 2.1: Map of UCL's Bloomsbury Campus extracted from OpenStreetMap data.

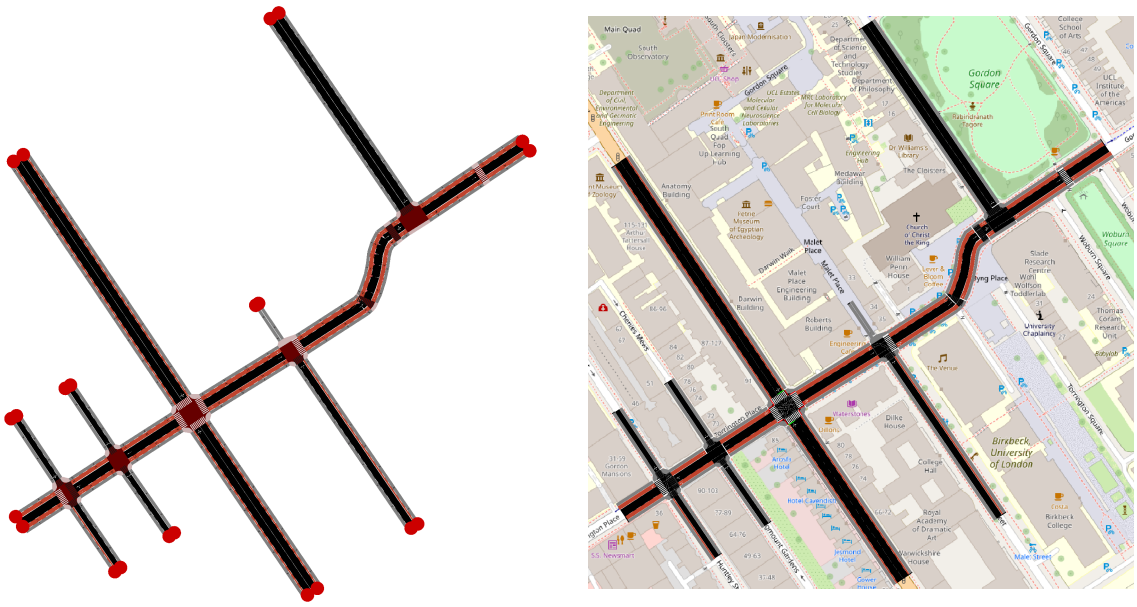
[1]The map centre is: 51.52519°N, 0.12870°W (WGS84).

Although the data from OpenStreetMap provides a lot of useful information, we will supplement it with Google Street View – featured in the Google Maps software – which provides interactive panoramas from positions along the streets. This will allows us to find out more precise information such as lane markings and widths.

2.2 Simulation of Urban MObility

Simulation of Urban MObility (SUMO) is an open source, multi-modal traffic simulation package designed to handle large networks. It allows for intermodal simulation including pedestrians and comes with a large set of tools for scenario creation.

Using the image in Figure 2.1, we can create a network using the *netedit* graphical network editor included in SUMO. Networks are primarily comprised of edges, which is subdivided into lanes, and junctions, which are nodes where the edges meet. Junctions additionally support crossings, lane connections, and traffic lights.



(a) Road network in netedit.

(b) Simulation view in SUMO.

Figure 2.2: Demo simulation environment.

In Figure 2.2a, the red circles and intermediary maroon links indicate junctions, whereas the in between sections are the edges. Each edge can be comprised of a bicycle lane, pedestrian lane, and/or vehicle lane as shown. Additionally, crossings have been added to the junctions, and a configurable traffic light system has been added with the lane connections appropriately adjusted.

Though not visible in the road geometry, lane and junction priorities have also been set, consistent with rules 170–183 of The Highway Code [DfT]. In the UK, 'main' and 'side' roads are relative terms, with the road with priority (e.g. no give-way markings, or priority road signs) functioning as the main road, while the road required to give way is the side road.

2.3 Route Generation

The SUMO installation includes an additional collection of tools for convenience. One such tool is a `randomTrips.py` script written in Python which enables automatic generation of trips and

routes[2] for different origin-destination pairs for a given road network.

We use this script to generate traffic for the following three types of entities, though SUMO provides the flexibility for more:

- Cars
- Bicycles
- Pedestrians

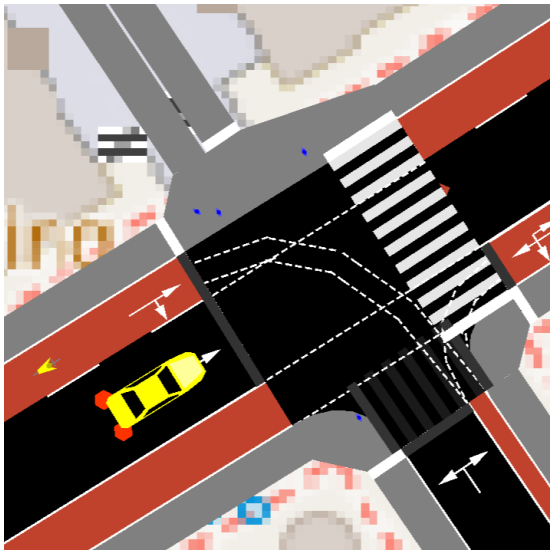


Figure 2.3: Example of car, bicycle, and pedestrian entities.

In Figure 2.3, the car, travelling east, is shown as a yellow rectangle with brake lights; the bicycle, travelling east on the bicycle lane, is depicted as a yellow trapezium-like shape emerging from the left side of the image; and the pedestrians are represented as blue circles.

Route generation is controlled by several parameters, including:

- `seed`: Ensures reproducibility of the random generation process.
- `insertion-density`: Controls the density of generated entities.
- `binomial`: Controls how vehicle insertions are distributed over time by sampling departures from a binomial distribution.
- `fringe-factor`: Ensures that vehicles spawn at the outer edges of the network.
- `random-factor`: Randomizes edge weights to increase variability between generated routes.

There are other obvious parameters omitted which allow you to specify information such as the input network for which the routes are generated, the output trip and route files, the entity type, and the start and end time of entity generation.

[2]N.B. a *trip* defines a random start A and end B, whereas a *route* defines the exact edges to traverse to get from that start A to end B.

2.3.1 Binomial Distribution

The `--binomial` parameter allows vehicle departures to be sampled from a Binomial distribution. Real-world traffic arrivals are not perfectly periodic due to the behaviour of drivers as well as variations in the environment. Rather than inserting vehicles into the simulation at perfectly regular intervals, stochastic variation is introduced such that some seconds contain more vehicle departures and others fewer.

$$X \sim \text{Binomial}(N, p)$$

$$\mathbb{E}[X] = Np$$

Where N is given by the `--binomial` parameter, and Np equals the insertion rate calculated from the `--insertion-density` parameter. Thus, the overall expected value of departures per time step still remains equal to the deterministic insertion rate.

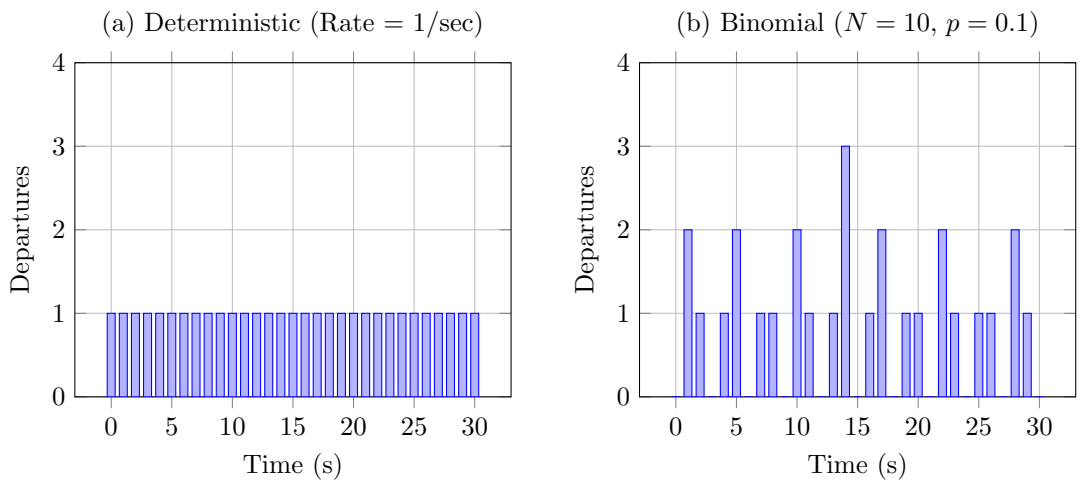


Figure 2.4: Comparison of deterministic and binomial vehicle insertions.

2.3.2 Poisson Distribution

In traffic theory, Poisson processes are commonly used and preferred over the Binomial distribution to model vehicle arrivals [GS55]. In fact, for sufficiently large N and small p, the binomial distribution approximates a Poisson distribution:

$$\text{Binomial}(N, p) \approx \text{Poisson}(\lambda), \quad \text{where} \quad \lambda = Np$$

The `randomTrips.py` script provides flexibility in defining vehicle traffic. For example, you may define there to be n fixed 'flows' which traffic may take – some of which may annoyingly be duplicates – or you let any of the possible routes for the traffic be arbitrarily, independently chosen each time a new vehicle is inserted.

Whilst there isn't a problem with the Poisson distribution per se, the script is designed such that it is only compatible with the prior definition method. Binomial sampling on the other hand is compatible with the latter method as well as other parameters which are more flexible for our purposes and have less implementation-specific issues. For this reason, it was chosen.

2.4 Summary

This chapter described the creation of a simulated environment based on the streets surrounding UCL's Bloomsbury campus. Road network data was extracted from OpenStreetMap and refined using Google Street View. The network was implemented using SUMO, where edges and junctions were defined.

Using SUMO's `randomTrips.py` tool, there is a way for scalable creation of diverse traffic scenarios. Stochastic vehicle insertions is used to better approximate real-world traffic variability. Overall, these components establish a way in which we can train and evaluate intelligent agents.

Chapter 3

Reward System

Now that we have seen how an environment – composed of a network and traffic routes – is constructed, we must calculate its baseline performance. We will comment on the environment specifics, including the current traffic light system algorithms, as well as how a reward is defined; a good reward system is paramount to training intelligent agents effectively.

3.1 Existing Traffic Lights

[WRITE ABOUT EXISTING TRAFFIC LIGHT SYSTEMS USED FOR THE BASELINE] ...

—

If you have a formal theorem you might try this.

DEFINITION 1 *See definition 1.*

THEOREM 2 *For all* $n \in \mathbb{N}$, $1^n = 1$.

PROOF:

By induction over n. $\square$

Chapter 4

etc.

Chapter 5

Conclusions

5.1 Achievements

Summarise the achievements to confirm the project goals have been met.

5.2 Evaluation

Evaluation of the work (this may be in a separate chapter if there is substantial evaluation).

5.3 Future Work

How the project might be continued, but don't give the impression you ran out of time!

Bibliography

[Pri70] A. Prior. The notion of the present. *Studium Generale*, 23: 245–248, 1970.

[Rey97] M. Reynolds. A decidable temporal logic of parallelism. *Notre Dame Journal of Formal Logic*, 38(3): 419–436, 1997.

[Tha17] Thames Leisure. 12 amazing facts about London. *Thames Leisure*, archived January 7, 2017. Available at: https://web.archive.org/web/20170107005431/http://www.thamesleisure.co.uk/12-amazing-facts-london/.

[FHW17] Federal Highway Administration. EDC-1: Adaptive Signal Control Technology. *Federal Highway Administration*, 2017. Available at: https://www.fhwa.dot.gov/innovation/everydaycounts/edc-1/asct.cfm.

[HW08] M. Haklay and P. Weber. OpenStreetMap: User-generated street maps. *IEEE Pervasive Computing*, 7(4): 12–18, 2008. doi:10.1109/MPRV.2008.80.

[DfT] Department for Transport. The Highway Code: Using the road (Rules 159–203). *GOV.UK*. Available at: https://www.gov.uk/guidance/the-highway-code/using-the-road-159-to-203. Accessed: 20 February 2026.

[GS55] D. L. Gerlough and A. Schuhl. Use of Poisson Distribution in Highway Traffic: The Probability Theory Applied to Distribution of Vehicles on Two-Lane Highways. *Research Paper, Eno Foundation for Highway Traffic Control*, 1955. Available at: https://rosap.ntl.bts.gov/view/dot/16299. Accessed: 26 February 2026.

Appendix A

Other appendices, e.g., code listing

Put your appendix sections here